

Tentu, mari kita bedah **kode C#** di Sobel.cs baris demi baris, fokus pada **logika pemrogramannya**.

Saya akan memecahnya menjadi 4 blok utama agar mudah dipahami.

BLOK 1: Persiapan & Akses Memori Cepat

Masalah utama pengolahan citra adalah: **Gambar itu besar**. Gambar 1080p punya 2 juta pixel. Kalau kita akses satu-satu pakai cara biasa, program bakal *lag*.

C#

```
// 1. Kunci gambar di memori (LockBits)
BitmapData srcData = bmp.LockBits(
    new Rectangle(0, 0, width, height),
    ImageLockMode.ReadOnly,
    PixelFormat.Format32bppArgb // Format 32-bit (Blue, Green, Red, Alpha)
);

// 2. Siapkan wadah (Array)
int bytes = Math.Abs(srcData.Stride) * height;
byte[] pixelBuffer = new byte[bytes]; // Wadah baca (Gambar Asli)
byte[] resultBuffer = new byte[bytes]; // Wadah tulis (Gambar Hasil)

// 3. Salin data dari Gambar ke Array
Marshal.Copy(srcData.Scan0, pixelBuffer, 0, bytes);
```

Penjelasan Detail:

- **LockBits:** Ini ibarat kita bilang ke Windows: *"Tolong jangan ganggu memori gambar ini, saya mau bongkar isinya."*
- **byte[]:** Kita mengubah gambar (yang 2 dimensi x,y) menjadi **deretan angka panjang (1 dimensi)**.
 - Bayangkan pixel itu seperti batu bata. Marshal.Copy merobohkan tembok bata itu menjadi satu baris panjang di tanah supaya kita bisa lari (looping) di atasnya dengan sangat cepat.
- **Kenapa ada 2 Array (pixelBuffer & resultBuffer)?**
 - PENTING: Kita **tidak boleh** menimpa data di pixelBuffer saat sedang menghitung.
 - Kalau kita ubah pixel di koordinat (0,0), nanti saat kita hitung pixel (0,1), tetangganya sudah berubah (sudah bukan data asli). Makanya kita baca dari A, tulis ke B.

BLOK 2: Definisi "Kacamata" (Kernel)

Ini adalah penerjemahan langsung dari matriks matematika ke Array C#.

C#

```
// Gx = Deteksi Vertikal
int[,] gx = {
    { -1, 0, 1 },
    { -2, 0, 2 }, // <-- Perhatikan angka 2 (Bobot lebih besar)
    { -1, 0, 1 }
};
// Gy = Deteksi Horizontal
int[,] gy = { ... };
```

Penjelasan Detail:

- `int[,]`: Array 2 dimensi.
- Ini adalah "cetakan" yang akan kita tempelkan ke setiap pixel nanti.

BLOK 3: Looping (Menyusuri Gambar)

Bagian ini menentukan pixel mana saja yang akan diproses.

C#

```
// Parallel.For = Gunakan semua inti CPU (Multithreading) biar cepat
Parallel.For(1, height - 1, y =>
{
    for (int x = 1; x < width - 1; x++)
    {
        // ... PROSES DI SINI ...
    }
});
```

Penjelasan Detail:

- **1 sampai height - 1**: Kenapa mulainya dari 1, bukan 0? Dan kenapa stop di width - 1?
 - Jawab: Karena Sobel butuh **tetangga kiri** dan **tetangga kanan**.
 - Pixel di koordinat 0 (paling pinggir kiri) **tidak punya tetangga kiri**.
 - Pixel di koordinat width (paling pinggir kanan) **tidak punya tetangga kanan**.
 - Jadi, kita sengaja **melewatkan bingkai luar** setebal 1 pixel agar program tidak crash (*Index Out of Range*).

BLOK 4: Inti Perhitungan (Konvolusi) - BAGIAN TERSULIT

Ini adalah tempat "magic" terjadi. Kita menempelkan matriks 3x3 ke pixel target.

C#

```
double sumX = 0;
double sumY = 0;

// Loop kecil 3x3 (ky = -1, 0, 1)
for (int ky = -1; ky <= 1; ky++)
{
    for (int kx = -1; kx <= 1; kx++)
    {
        // RUMUS MENCARI POSISI PIXEL TETANGGA DALAM ARRAY 1D
        int pos = (y + ky) * stride + (x + kx) * 4;

        // Ambil warnanya
        int val = pixelBuffer[pos];

        // Kalikan warna dengan angka di matriks Kernel
        // gx[ky + 1, kx + 1] -> Kita tambah +1 karena array mulai dari 0, bukan -1
        sumX += val * gx[ky + 1, kx + 1];
        sumY += val * gy[ky + 1, kx + 1];
    }
}
```

Penjelasan Rumus pos:

`int pos = (y + ky) * stride + (x + kx) * 4;`

- Kita sedang mencari alamat pixel tetangga di memori.
- `(y + ky) * stride`:
 - `y`: Baris saya sekarang.

- ky: Geser baris (-1 berarti baris atas, +1 baris bawah).
- stride: Lebar satu baris gambar dalam byte. Jadi ini artinya "Lompat sekian baris ke bawah".
- $(x + kx) * 4$:
 - x: Kolom saya sekarang.
 - kx: Geser kolom (-1 kiri, +1 kanan).
 - * 4: Karena 1 pixel = 4 byte (Blue, Green, Red, Alpha). Kita harus lompat 4 langkah memori untuk pindah 1 pixel.

BLOK 5: Matematika Pythagoras & Penyimpanan

Setelah loop 3x3 selesai, kita punya sumX (total kekuatan vertikal) dan sumY (total kekuatan horizontal).

C#

```
// 1. Gabungkan X dan Y (Rumus Pythagoras)
int magnitude = (int)Math.Sqrt((sumX * sumX) + (sumY * sumY));

// 2. Clamping (Potong nilai yang kelewatan)
if (magnitude > 255) magnitude = 255;
if (magnitude < 0) magnitude = 0;

// 3. Simpan ke Buffer Hasil
// Kita cari alamat pixel tujuan (k)
int k = y * stride + x * 4;

resultBuffer[k] = (byte)magnitude; // Blue (Diisi abu-abu)
resultBuffer[k + 1] = (byte)magnitude; // Green (Diisi abu-abu)
resultBuffer[k + 2] = (byte)magnitude; // Red (Diisi abu-abu)
resultBuffer[k + 3] = 255; // Alpha (Full, tidak transparan)
```

Penjelasan Detail:

- **magnitude**: Jika magnitude diisi ke R, G, dan B dengan angka yang sama, hasilnya adalah warna **Grayscale** (Abu-abu).
 - Jika 0 (Hitam) -> Bukan Tepi.
 - Jika 255 (Putih) -> Tepi Kuat.
- **resultBuffer[k + 3] = 255**: Ini channel Alpha (Transparansi). Kalau kita lupa isi ini (nilainya 0), gambarnya akan jadi transparan (hilang). Kita isi 255 biar gambarnya muncul.

Ringkasan untuk Ujian (Bila ditanya dosen soal kode):

1. **Kenapa pakai LockBits?** Biar cepat, Pak. Akses array langsung jauh lebih cepat daripada SetPixel.
2. **Kenapa loop mulai dari x=1?** Karena Sobel butuh tetangga kiri. Pixel paling kiri (x=0) tidak punya tetangga kiri, jadi dilewatkan.
3. **Apa itu stride?** Itu lebar satu baris data gambar di memori.
4. **Kenapa * 4?** Karena format gambarnya 32-bit (ARGB), jadi satu pixel makan 4 kotak memori (byte).

Sudah cukup jelas bagian teknis kodenya?